

MPI Programming Tutorial

Parallel Programming on Your Own Laptop
Linux or a Linux Virtual Machine in VMware

Department of Computer Science and Electronics
Universitas Gadjah Mada

How to use this tutorial

This tutorial teaches message-passing programming in C using Open MPI. Complete the main exercises on one Linux system: either Linux installed directly on your laptop or a Linux guest running inside VMware. One virtual machine is enough to run multiple MPI processes. An optional exercise later distributes one program across two virtual machines.

Prerequisites: basic C programming, arrays, loops, functions, pointers, and basic terminal use. You do not need a server, GPU, cluster, or job scheduler. Allow several laboratory sessions to complete all exercises.

Learning outcomes. After completing the tutorial, you should be able to:

1. Compile and launch MPI programs with different process counts.
2. Explain ranks, communicators, and separate process memory.
3. Implement point-to-point communication without circular waiting.
4. Distribute data and combine results using collective communication.
5. Use nonblocking operations with correct buffer lifetimes.
6. Measure parallel execution time and interpret speedup.

Working method. For every exercise: predict the result, create the C file, compile it, run it, compare the output with your prediction, and answer the checkpoint. Copy C code from this source file when possible; copying from a PDF can introduce formatting errors. Every complete C listing is a separate program. Shell listings contain commands to run in the Linux terminal.

Overleaf only compiles this document. Compile and execute the C programs inside Linux, not inside Overleaf.

Contents

1	Understand the execution model	3
2	Prepare your Linux environment	4
3	Exercise 1: hello, rank, and private memory	5
4	Exercise 2: send a value and receive a reply	6
5	Exercise 3: deadlock and a ring exchange	7
6	Collective communication: choose the operation	9
7	Exercise 4: broadcast a configuration value	9
8	Exercise 5: scatter, compute, gather, and reduce	10
9	Exercise 6: make a result available everywhere	12
10	Exercise 7: nonblocking communication	13
11	Exercise 8: parallel numerical integration	14
12	Measure and interpret performance	16
13	Optional: run one job across two VMware guests	17
14	Troubleshooting	19
15	Practice tasks and report	19

1 Understand the execution model

1.1 Essential vocabulary

Term	Meaning in this tutorial
MPI	Message Passing Interface: a standard for communication between processes. It is not a programming language.
Open MPI	An implementation of MPI, providing libraries, compiler wrappers, and tools for launching programs.
Process	A running instance of a program with its own address space.
Rank	An integer identifying a process within a communicator. With P processes, ranks are $0, \dots, P - 1$.
Communicator	A communication context and a group of processes. <code>MPI_COMM_WORLD</code> contains all processes in the jobs used here.
Size	The number of processes in a communicator.
Node	A machine participating in a job. A VM can act as a node.
Core / vCPU	A physical CPU core / a virtual CPU exposed to a VM. Neither is the same thing as an MPI rank.
Root	A rank selected for a particular collective operation. It is often rank 0, but this is a programming choice.
Hostname	The operating system's name for a machine; several ranks can report the same hostname.

1.2 What happens when a program starts?

```
mpirun -np 4 ./hello
```

This launches four processes running the same executable. Each process starts at `main`, initializes MPI, and obtains its own rank. They can execute different branches based on rank. This pattern is called *single program, multiple data* (SPMD).

On your standalone laptop, this command launches the processes locally. It does not discover other laptops automatically. Using additional machines requires explicit launch configuration or a cluster scheduler.

MPI has no mandatory master–worker hierarchy. Rank 0 is an ordinary process unless your program gives it a special role. Ranks do not imply execution order, priority, or a permanent mapping to cores.

1.3 Separate memory

If every process declares `int x = 0;`, each has a separate `x`. Changing rank 0's variable does not change rank 1's variable. This also applies to ordinary global and static C variables. In these exercises, values become available elsewhere through explicit MPI communication. MPI has advanced shared-memory and remote-memory features, but those features are outside this tutorial.

Checkpoint. If four ranks run on one laptop, how many processes and how many nodes are involved? Can they have different values for a variable with the same name?

Reference: [MPI Forum: MPI standard](#).

2 Prepare your Linux environment

2.1 Option A: Linux installed directly

Open a terminal and continue to Section 2.3. The package commands below target Ubuntu/Debian. On another distribution, install the equivalent C development tools and Open MPI packages using its package manager.

2.2 Option B: Linux inside VMware

1. Install a VMware desktop product compatible with your host computer, or use your existing installation.
2. Download an Ubuntu installation ISO from [the official Ubuntu website](#). Ubuntu 24.04 LTS is one suitable baseline for this tutorial.
3. Create a new VM and attach the ISO as its installation media. Use an ISO architecture supported by your computer and VMware product.
4. As a starting configuration, allocate 2 vCPUs, 4 GB RAM, and a 25 GB virtual disk. Leave sufficient CPU and RAM for the host system. A lightweight/server installation can reduce memory use.
5. Select NAT networking for the initial one-VM exercises. NAT lets the guest access the Internet through the host.
6. Install Ubuntu and create a normal user account. After installation, boot from the virtual disk and open a Linux terminal.

All installation, compilation, and execution commands below run **inside the Linux guest**. The main exercises do not require SSH or VM-to-VM networking. Increasing the VM's vCPU count does not create additional physical CPU cores.

2.3 Install the tools

```
sudo apt update
sudo apt install build-essential openmpi-bin libopenmpi-dev

gcc --version
mpicc --showme:version
mpirun --version
nproc
hostname

mkdir -p ~/mpi-lab
cd ~/mpi-lab
```

- `build-essential` supplies basic compilation tools.
- `openmpi-bin` supplies Open MPI runtime commands.
- `libopenmpi-dev` supplies headers and development libraries.
- `mpicc` wraps a C compiler and adds MPI compilation/linking options.
- `mpirun` launches a job; `-np` specifies its process count.
- `nproc` reports processing units available to the current process, which need not equal the number of physical cores or Open MPI slots.

Use a text editor to create each file, for example `nano hello.c`. In nano, press Ctrl+O and Enter to save, then Ctrl+X to exit. Run MPI programs as your normal user; do not put `sudo` before `mpirun`.

Reference: [Ubuntu: Open MPI packages](#).

3 Exercise 1: hello, rank, and private memory

Save the following complete program as `hello.c`.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank, size, name_len;
    char name[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &name_len);

    int x = 0;
    if (rank == 0)
        x = 100;

    printf("rank=%d size=%d host=%s x=%d\n",
           rank, size, name, x);

    MPI_Finalize();
    return 0;
}
```

```
mpicc -std=c11 -O2 -Wall -Wextra hello.c -o hello
mpirun -np 1 ./hello
mpirun -np 2 ./hello
mpirun -np 4 ./hello
```

`MPI_Init` initializes MPI. `MPI_Comm_rank` and `MPI_Comm_size` write values into the variables whose addresses you provide. `MPI_Finalize` ends this program's use of MPI; finish all pending communication before calling it.

The compiler options select C11, enable optimization, and enable useful warnings. `-o hello` names the executable; `./hello` refers to that executable in the current directory.

One possible four-process output is:

```
rank=2 size=4 host=ubuntu x=0
rank=0 size=4 host=ubuntu x=100
rank=3 size=4 host=ubuntu x=0
rank=1 size=4 host=ubuntu x=0
```

The line order is not guaranteed. Independent processes and output forwarding affect the order in which text reaches the terminal. Rank 0 does not automatically execute first.

3.1 If four processes exceed the available slots

```
mpirun --oversubscribe -np 4 ./hello
```

Oversubscription allows more processes than the launcher's available slots. A slot represents process capacity used for placement. This option is useful for functional exercises on a small VM, but extra

processes can compete for the same CPUs and reduce performance. Use it only when necessary.

Reference: [Open MPI: process scheduling and oversubscription](#).

Checkpoint. Explain why only rank 0 prints `x=100`. Run the program three times and describe whether the line order changes.

4 Exercise 2: send a value and receive a reply

4.1 Understand the message before writing the program

`MPI_Send` sends data from a process's memory; `MPI_Recv` receives data into another process's memory.

Argument	Meaning
Buffer	Address of the first data element, such as <code>&value</code> .
Count	Number of elements, not the number of bytes.
Datatype	MPI's description of each element, such as <code>MPI_INT</code> .
Peer rank	Destination/source in the specified communicator.
Tag	An integer label chosen by the program to distinguish messages.
Communicator	The communication context, here <code>MPI_COMM_WORLD</code> .
Status	Receive metadata; use <code>MPI_STATUS_IGNORE</code> if it is unneeded.

A receive matches a message addressed to that process with a compatible source, tag, and communicator. Datatypes must be compatible and the receive buffer must be large enough. A datatype mismatch is an error, not a way to select a different message.

Reference: [Open MPI: MPI_Recv](#).

4.2 Complete program

Save as `request_reply.c`. Run with exactly two processes.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0)
            fprintf(stderr, "Use exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        int value = 21, reply = 0;
        MPI_Send(&value, 1, MPI_INT, 1, 10,
                 MPI_COMM_WORLD);
        MPI_Recv(&reply, 1, MPI_INT, 1, 20,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 0 sent %d and received %d\n",
               value, reply);
    }
}
```

```

    } else {
        int value = 0;
        MPI_Recv(&value, 1, MPI_INT, 0, 10,
                MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        int reply = 2 * value;
        MPI_Send(&reply, 1, MPI_INT, 0, 20,
                MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}

```

```

mpicc -std=c11 -O2 -Wall -Wextra request_reply.c -o request_reply
mpirun -np 2 ./request_reply

```

Rank 0 sent 21 and received 42

4.3 What does blocking actually guarantee?

- After `MPI_Send` returns, the sender may reuse its send buffer. This does not prove that the receiver has processed the value.
- A standard send may finish using internal buffering, or may wait for a matching receive. Correctness must not depend on buffering being available.
- After `MPI_Recv` returns, the received data is available in the receive buffer. Until then, this process waits at that call.
- In this program, receiving the reply provides application-level evidence that rank 1 received and processed the request.

Reference: [MPI Forum: communication modes](#).

Checkpoint. Change the request to 35. Predict the reply. What happens if the first receive expects tag 11 while the send uses tag 10? If you try this intentional error, stop the waiting job with Ctrl+C.

5 Exercise 3: deadlock and a ring exchange

5.1 How circular waiting occurs

A *deadlock* occurs when processes wait for events that cannot occur. Consider this incorrect fragment executed by both ranks in a two-process job:

```

/* Incorrect fragment: both processes receive first. */
int peer = 1 - rank;
MPI_Recv(&incoming, 1, MPI_INT, peer, 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Send(&outgoing, 1, MPI_INT, peer, 0,
        MPI_COMM_WORLD);

```

Neither process reaches its send. Both remain waiting at the receive.

Reversing both calls is not a general solution. If both ranks call `MPI_Send` first, a small message may appear to work because of buffering, while another message size or implementation may cause circular waiting. One solution for two ranks is asymmetric ordering: one sends first and the other

receives first, as in Exercise 2.

Reference: MPI Forum: point-to-point communication semantics.

5.2 Exchange with MPI_Sendrecv

MPI_Sendrecv combines a send and a receive in one blocking call. It supports an exchange without requiring all processes to finish their send before posting a receive. Correct peer, tag, and buffer choices are still required. Use separate send and receive buffers for this routine.

Reference: Open MPI: MPI_Sendrecv.

Save as ring.c.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int right = (rank + 1) % size;
    int left = (rank - 1 + size) % size;
    int outgoing = rank;
    int incoming = -1;

    MPI_Sendrecv(&outgoing, 1, MPI_INT, right, 0,
                &incoming, 1, MPI_INT, left, 0,
                MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    printf("rank=%d sent=%d to=%d received=%d from=%d\n",
           rank, outgoing, right, incoming, left);
    MPI_Finalize();
    return 0;
}
```

```
mpicc -std=c11 -O2 -Wall -Wextra ring.c -o ring
mpirun -np 4 ./ring
```

With four ranks, the logical ring is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$. Ranks 0, 1, 2, and 3 receive values 3, 0, 1, and 2, respectively. This logical structure does not imply four physical machines.

Should every program use Sendrecv? It is convenient for paired exchanges. Separate send/receive calls suit one-way transfers and request-reply protocols. Nonblocking calls allow multiple operations to be outstanding. Collective operations directly express communication involving a whole group.

Checkpoint. Run with one, two, and four processes. Explain why the left neighbor is computed as $(\text{rank} - 1 + \text{size}) \bmod \text{size}$.

6 Collective communication: choose the operation

A *collective* involves all processes in a communicator. The following descriptions apply to `MPI_COMM_WORLD` in these exercises.

Operation	Purpose
<code>MPI_Bcast</code>	Copy the root's data to every rank, including the root.
<code>MPI_Scatter</code>	Distribute equal-sized blocks of the root's array.
<code>MPI_Gather</code>	Collect equal-sized contributions into the root's array, in rank order.
<code>MPI_Reduce</code>	Combine contributions using an operation such as sum; place the result at the root.
<code>MPI_Allreduce</code>	Combine contributions and make the result available at every rank.
<code>MPI_Barrier</code>	Wait until all ranks in the communicator have entered the barrier. It transfers no application data.

Participation rules. All ranks must call matching collective operations in the same order. For rooted collectives, they must agree on the root. Argument roles can differ between root and non-root ranks. A collective is not automatically a barrier: its return does not generally mean that all other processes have returned.

Reference: [MPI Forum: collective communication and correctness](#).

This is incorrect:

```
/* Incorrect: other ranks never participate. */
if (rank == 0)
    MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

This is correct:

```
if (rank == 0)
    value = 25;
MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Only initialization belongs inside the rank-0 branch. The broadcast call belongs on every rank's execution path.

7 Exercise 4: broadcast a configuration value

Imagine that rank 0 selects the number of iterations for a simulation. Every rank needs the same value. Save as `broadcast.c`.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int iterations = 0;
    if (rank == 0)
        iterations = 1000;

    MPI_Bcast(&iterations, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```

    printf("rank=%d iterations=%d\n", rank, iterations);

    MPI_Finalize();
    return 0;
}

```

```

mpicc -std=c11 -O2 -Wall -Wextra broadcast.c -o broadcast
mpirun -np 4 ./broadcast

```

Every rank should print `iterations=1000`. Each still owns a separate variable; the broadcast copied a value into those separate variables.

Reference: [Open MPI: MPI_Bcast](#).

Checkpoint. Modify the program so rank 1 initializes the value and becomes the root. Run with at least two processes. Which two places must change?

8 Exercise 5: scatter, compute, gather, and reduce

8.1 Plan the data distribution

Compute the sum of integers 1 through 12. With four processes, each receives three consecutive elements:

Rank	Local elements	Local sum
0	1, 2, 3	6
1	4, 5, 6	15
2	7, 8, 9	24
3	10, 11, 12	33

`MPI_Gather` will collect [6,15,24,33] at rank 0. `MPI_Reduce` with `MPI_SUM` will instead combine those values into 78. The program performs both operations to show their difference.

8.2 Complete program

Save as `scatter_sum.c`.

```

#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    enum { N = 12 };
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size > N || N % size != 0) {
        if (rank == 0)
            fprintf(stderr, "Use 1, 2, 3, 4, 6, or 12 ranks.\n");
        MPI_Finalize();
        return 1;
    }
}

```

```

/* Only rank 0 initializes the full input. */
int data[N] = {0};
if (rank == 0)
    for (int i = 0; i < N; ++i)
        data[i] = i + 1;

int count = N / size;
int local[N] = {0};
MPI_Scatter(data, count, MPI_INT,
            local, count, MPI_INT, 0, MPI_COMM_WORLD);

int local_sum = 0;
for (int i = 0; i < count; ++i)
    local_sum += local[i];

int partials[N] = {0};
MPI_Gather(&local_sum, 1, MPI_INT,
          partials, 1, MPI_INT, 0, MPI_COMM_WORLD);

int total = 0;
MPI_Reduce(&local_sum, &total, 1, MPI_INT,
           MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    printf("Gathered partial sums:");
    for (int r = 0; r < size; ++r)
        printf(" %d", partials[r]);
    printf("\nReduced total: %d\n", total);
}

MPI_Finalize();
return 0;
}

```

```

mpicc -std=c11 -O2 -Wall -Wextra scatter_sum.c -o scatter_sum
mpirun -np 4 ./scatter_sum
mpirun -np 3 ./scatter_sum

```

Expected output for four processes:

```

Gathered partial sums: 6 15 24 33
Reduced total: 78

```

8.3 Read the counts carefully

In `MPI_Scatter`, `count` is the number of input elements sent *to each rank*; it is not 12 in the four-process case. The root also receives its own block.

Reference: [Open MPI: MPI_Scatter](#).

In `MPI_Gather`, each rank sends one partial sum, so the root's receive count is also 1 *per rank*. The receive array needs capacity for `size` integers. Only the root's gather receive buffer is significant. The small fixed arrays on every rank keep this introductory program simple.

Reference: [Open MPI: MPI_Gather](#).

Only rank 0 receives the valid global result from this `MPI_Reduce`. Here each rank contributes one

integer and `MPI_SUM` specifies addition.

Reference: [Open MPI: MPI_Reduce](#).

Uneven data sizes. This exercise deliberately requires 12 to be divisible by the process count. For an arbitrary array size, use `MPI_Scatterv` with per-rank counts and offsets, or another explicit partitioning strategy. Do not silently discard leftover elements.

Checkpoint. Predict the partial sums for three processes. Explain why replacing `MPI_Gather` with `MPI_Reduce` would lose information about individual ranks.

9 Exercise 6: make a result available everywhere

An iterative algorithm may require every rank to know the global error or global sum. Use `MPI_Allreduce` when all ranks need the combined result. There is no root argument. Save as `allreduce.c`.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double local = rank + 1.0;
    double global_sum = 0.0;
    MPI_Allreduce(&local, &global_sum, 1, MPI_DOUBLE,
                 MPI_SUM, MPI_COMM_WORLD);

    double mean = global_sum / size;
    printf("rank=%d sum=%.1f mean=%.1f\n",
           rank, global_sum, mean);

    MPI_Finalize();
    return 0;
}
```

```
mpicc -std=c11 -O2 -Wall -Wextra allreduce.c -o allreduce
mpirun -np 4 ./allreduce
```

With four ranks, every rank prints `sum=10.0 mean=2.5`. This example averages four rank-level values. If ranks hold different numbers of samples, compute the global sum and global sample count; averaging local means directly can give an incorrect result.

Reference: [Open MPI: MPI_Allreduce](#).

Checkpoint. Change `MPI_SUM` to `MPI_MAX`. Remove the mean calculation and report the global maximum. Predict the result first.

10 Exercise 7: nonblocking communication

`MPI_Irecv` and `MPI_Isend` initiate operations and return request handles. Returning from these calls does not mean that the transfers have completed. `MPI_Waitall` waits for the listed requests to complete.

While an operation is pending, do not modify its send buffer and do not access its receive buffer. Also keep both buffers allocated until completion. Independent computation can take place in between initiation and completion, although useful overlap depends on the MPI implementation and workload.

Reference: Open MPI: `MPI_Isend`.

Save as `nonblocking_ring.c`.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int left = (rank - 1 + size) % size;
    int right = (rank + 1) % size;
    int outgoing = rank, incoming = -1;
    MPI_Request requests[2];

    MPI_Irecv(&incoming, 1, MPI_INT, left, 0,
             MPI_COMM_WORLD, &requests[0]);
    MPI_Isend(&outgoing, 1, MPI_INT, right, 0,
             MPI_COMM_WORLD, &requests[1]);

    /* Independent work: neither buffer is accessed. */
    double work = 0.0;
    for (int i = 1; i <= 10000; ++i)
        work += 1.0 / (i + rank + 1.0);

    MPI_Waitall(2, requests, MPI_STATUSES_IGNORE);
    printf("rank=%d received=%d work=%.6f\n",
           rank, incoming, work);

    MPI_Finalize();
    return 0;
}
```

```
mpicc -std=c11 -O2 -Wall -Wextra nonblocking_ring.c -o nonblocking_ring
mpirun -np 4 ./nonblocking_ring
```

The received values should match Exercise 3. The loop illustrates a place for independent work; this small program does not establish a performance advantage. Posting a nonblocking send and immediately waiting for it before posting a needed receive can reintroduce circular waiting.

Checkpoint. Why would assigning a new value to `outgoing` before `MPI_Waitall` be incorrect? Why must printing `incoming` wait?

11 Exercise 8: parallel numerical integration

11.1 Define the task

Approximate π using the midpoint rule:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx \frac{1}{N} \sum_{i=0}^{N-1} \frac{4}{1 + ((i + 0.5)/N)^2}.$$

Each index corresponds to one independent function evaluation. We distribute contiguous index ranges and reduce their partial sums.

For P processes, let $q = \lfloor N/P \rfloor$ and $r = N \bmod P$. Rank k handles

$$n_k = q + \begin{cases} 1, & k < r, \\ 0, & k \geq r, \end{cases} \quad s_k = kq + \min(k, r).$$

Its indices run from s_k through $s_k + n_k - 1$. This partition includes every index exactly once, even when N is not divisible by P . For $N = 10$, $P = 3$, the ranges are $[0, 4)$, $[4, 7)$, and $[7, 10)$, where the upper endpoint is excluded.

11.2 Complete program

Save as `pi_mpi.c`. Only rank 0 parses the optional command-line input, then broadcasts the chosen value.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <math.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    long long n = 10000000;
    int valid = 1;
    if (rank == 0) {
        if (argc > 2)
            valid = 0;
        if (argc == 2) {
            char *end = NULL;
            errno = 0;
            n = strtoll(argv[1], &end, 10);
            if (errno != 0 || end == argv[1] || *end != '\0'
                || n < 1 || n > 1000000000LL)
                valid = 0;
        }
    }

    MPI_Bcast(&valid, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (!valid) {
        if (rank == 0)
```

```

        fprintf(stderr,
            "Usage: ./pi_mpi [N: 1..1000000000]\n");
    MPI_Finalize();
    return 1;
}
MPI_Bcast(&n, 1, MPI_LONG_LONG_INT, 0, MPI_COMM_WORLD);

long long q = n / size, rem = n % size;
long long count = q + (rank < rem ? 1 : 0);
long long start = rank * q + (rank < rem ? rank : rem);
long long end = start + count;
double h = 1.0 / (double)n;

MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();

double local_sum = 0.0;
for (long long i = start; i < end; ++i) {
    double x = ((double)i + 0.5) * h;
    local_sum += 4.0 / (1.0 + x * x);
}
double local_pi = h * local_sum;
double pi = 0.0;
MPI_Reduce(&local_pi, &pi, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);

double elapsed = MPI_Wtime() - t0;
double max_elapsed = 0.0;
MPI_Reduce(&elapsed, &max_elapsed, 1, MPI_DOUBLE,
           MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0) {
    double reference = acos(-1.0);
    printf("N=%lld processes=%d pi=%.15f\n",
        n, size, pi);
    printf("absolute_error=%.3e time_seconds=%.6f\n",
        fabs(pi - reference), max_elapsed);
}

MPI_Finalize();
return 0;
}

```

```

mpicc -std=c11 -O2 -Wall -Wextra pi_mpi.c -lm -o pi_mpi
mpirun -np 1 ./pi_mpi 10000000
mpirun -np 2 ./pi_mpi 10000000
mpirun -np 4 ./pi_mpi 10000000

```

The `-lm` option links the mathematics library. For large N , the estimate should be close to 3.141592653589793. The last digits may vary with process count because floating-point addition is not exactly associative. For $N = 1$, the estimate is exactly 3.2 in real arithmetic; small N is useful for checking the partition, not accuracy.

11.3 Understand the measurement

`MPI_Wtime` returns local wall-clock time in seconds. Compute a duration using two timestamps from the same rank; clocks on different nodes need not be synchronized.

Reference: Open MPI: MPI_Wtime.

The initial barrier places ranks at the start of the measured phase. The measured interval includes the local numerical computation and the first reduction. The second reduction reports the maximum of the measured local durations. This is a practical approximation to parallel phase time. It excludes MPI startup, input broadcast, the initial barrier, the timing reduction, printing, and finalization.

Checkpoint. Run with $N = 10$ and three processes, then $N = 2$ and four processes. Explain how ranks with zero assigned iterations still participate correctly in the reductions.

12 Measure and interpret performance

12.1 Experimental procedure

1. Record the CPU model, physical core count, operating system, Open MPI version, and whether Linux runs natively or in a VM. For a VM, record its allocated vCPUs and RAM.
2. Select a fixed N . Start with 10^7 ; increase it within the program's limit if timings are too short or unstable.
3. Use the same executable and optimization flags for every run. Keep the laptop plugged in and minimize background activity.
4. Run one warm-up and then at least five measured runs per process count. Compare one, two, and, if resources permit, four processes.
5. Report the median time and the minimum–maximum range. Label any oversubscribed runs separately.

```
lscpu
mpirun --version

# Example: one warm-up and five measured runs with two ranks.
mpirun -np 2 ./pi_mpi 10000000
for trial in 1 2 3 4 5
do
    mpirun -np 2 ./pi_mpi 10000000
done
```

12.2 Speedup and efficiency

Let T_1 be the median measured phase time with one MPI process and T_P the time with P processes:

$$S_P = \frac{T_1}{T_P}, \quad E_P = \frac{S_P}{P}.$$

This uses the one-process MPI program as its baseline. It is not a comparison against a separately optimized serial implementation. Holding N fixed while changing P is called *strong scaling*.

Processes	Median time (s)	Range (s)	Speedup	Efficiency
1	_____	_____	1.00	1.00
2	_____	_____	_____	_____
4	_____	_____	_____	_____

For example, if measured times were $T_1 = 0.80$ s and $T_4 = 0.25$ s, the speedup would be 3.2 and the efficiency 0.8, or 80%. These numbers illustrate the calculation; they are not benchmark results.

12.3 Why more processes can be slower

Communication, synchronization, CPU competition, memory bandwidth, scheduling, and VM overhead can offset the benefit of dividing the work. Very small tasks may spend more time coordinating than computing. Two VMs on the same laptop still share the same physical hardware.

For a simplified workload with serial fraction f , Amdahl's law predicts

$$S_P \leq \frac{1}{f + (1 - f)/P},$$

before additional communication and other overheads are included. Use this model to reason about limits, not to assume every measured program follows it exactly.

Checkpoint. Explain an observed slowdown using your measurements. Distinguish program correctness from performance improvement.

13 Optional: run one job across two VMware guests

13.1 Goal and preparation

Use this extension only after the local exercises work. It teaches remote launch and distributed execution; it does not add physical computing resources to your laptop.

Prepare two Linux VMs with the same architecture, Ubuntu release, Open MPI version, and username. In the examples the username is `student`; replace it everywhere with your actual username. Use distinct hostnames, such as `mpi1` and `mpi2`. Install the MPI tools from Section 2.3 on both guests.

Connect both guests to the **same VMware NAT virtual network**. Each guest needs a distinct IP address reachable from the other. One virtual network adapter per guest keeps this exercise simple. The exact VMware menu labels depend on the installed product.

On each VM, inspect its address and start an SSH server:

```
ip -br addr
sudo apt install openssh-server
sudo systemctl enable --now ssh
mkdir -p ~/mpi-lab
```

For illustration, assume VM 1 has `192.168.150.101` and VM 2 has `192.168.150.102`. **Replace these addresses with the real ones** in every following command and hostfile. Recheck them after VM restarts.

13.2 Configure noninteractive SSH

SSH allows the launcher to start processes remotely. For this disposable private lab, generate a separate key on each VM. Use the proposed filename only if it does not already exist; do not overwrite an existing key. Keep the key inside the lab VMs.

On each VM:

```
ssh-keygen -t ed25519 -f ~/.ssh/mpi_lab -N ""
ssh-copy-id -i ~/.ssh/mpi_lab.pub student@192.168.150.101
ssh-copy-id -i ~/.ssh/mpi_lab.pub student@192.168.150.102
```

Verify the SSH host fingerprint when first prompted. The account password may be requested while installing each public key. Add the following host-specific entry to `~/.ssh/config` on each VM, preserving any existing configuration:

```
Host 192.168.150.101 192.168.150.102
    User student
    IdentityFile ~/.ssh/mpi_lab
    IdentitiesOnly yes
```

Then run on **each** VM:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/config ~/.ssh/mpi_lab
ssh -o BatchMode=yes student@192.168.150.101 hostname
ssh -o BatchMode=yes student@192.168.150.102 hostname
```

Both commands must return hostnames without prompting. This includes connections to the VM itself. Restrict these passphrase-free keys to the lab; remove the relevant authorized public keys when retiring the VMs.

Reference: [Open MPI: launching through SSH](#).

13.3 Deploy the same program

The executable must exist at the same absolute path on both guests, with compatible libraries. The hostfile does not distribute program files. From **VM 1**, where `hello.c` is already in `~/mpi-lab`:

```
cd ~/mpi-lab
mpicc -std=c11 -O2 -Wall -Wextra hello.c -o hello
scp hello.c student@192.168.150.102:/home/student/mpi-lab/
ssh student@192.168.150.102 \
    'cd /home/student/mpi-lab && mpicc -std=c11 -O2 -Wall -Wextra hello.c -o hello'
ssh student@192.168.150.102 'mpirun --version'
```

Create `hosts.txt` in `~/mpi-lab` on VM 1:

```
192.168.150.101 slots=2
192.168.150.102 slots=2
```

Use two slots per guest only if each guest has two vCPUs available for this exercise. For one vCPU per guest, use `slots=1`, two total processes, and `ppr:1:node` in the next command.

Launch once, from **VM 1**:

```
mpirun --hostfile hosts.txt --map-by ppr:2:node -np 4 \
    /home/student/mpi-lab/hello
```

`ppr:2:node` requests two processes per node. The output should contain four ranks across two hostnames. Do not run a second independent launcher on VM 2.

SSH is used for remote launch; MPI communication also needs direct connectivity between the guests. If a firewall is enabled, allow communication from the peer guest on this private lab network rather than disabling the firewall globally. For example, on VM 1 with UFW:

```
sudo ufw allow from 192.168.150.102
```

On VM 2, the corresponding peer address is 192.168.150.101. Apply such rules only for your actual private lab addresses.

Checkpoint. Verify two distinct hostnames. Then deploy `ring.c` to both VMs and repeat its exchange. Explain which parts of the C program changed.

14 Troubleshooting

Symptom	Check or corrective action
<code>mpicc</code> not found	Install the MPI development packages inside the Linux system where you are compiling. Check which <code>mpicc</code> .
<code>mpi.h</code> not found	Compile using <code>mpicc</code> , and verify that <code>libopenmpi-dev</code> is installed.
Undefined MPI references	Use <code>mpicc</code> for the link step as well as compilation.
Not enough slots	Use fewer processes, or use <code>-oversubscribe</code> for a functional exercise.
Refusal to run as root	Run the program from a normal user account without <code>sudo</code> .
Program appears stuck	Check matching ranks, tags, communicators, call ordering, and participation in collectives. Stop an intentional deadlock with Ctrl+C.
Invalid rank	Ensure the destination/source is in the communicator's rank range. Check the required <code>-np</code> value.
Truncated message	The receive buffer/count is too small for the matching message. Check element counts and datatypes.
Unexpected output order	This is normal for independent ranks. For ordered reporting, gather data and print at one rank; a barrier alone does not order forwarded terminal output.
Remote SSH asks for a password	Check public-key installation, key permissions, SSH configuration, and noninteractive access from every guest.
Remote executable not found	Deploy the file on every node and use an identical absolute path.
SSH works but MPI hangs	Check peer-to-peer networking, firewall rules, MPI version consistency, and whether extra network interfaces are causing unreachable routes.
More ranks are slower	Check workload size, available CPUs, oversubscription, communication costs, and VM resource contention.

15 Practice tasks and report

15.1 Tasks

1. **Array request-reply.** Extend Exercise 2 to send five integers to rank 1. Rank 1 squares each element and returns the five results. Use a small input such as [1, 2, 3, 4, 5], whose result you can check manually.
2. **Distributed statistics.** Extend Exercise 5 to calculate the global sum and maximum of the input. Explain why these are reductions and identify the reduction operators.
3. **Uneven work.** For $N = 17$ and $P = 4$, list each rank's start index and number of iterations using Exercise 8's partitioning rule. Check for gaps and overlap.
4. **Performance study.** Measure Exercise 8 with at least two values of N and the process counts supported by your laptop. Explain when additional processes help.
5. **Optional distributed execution.** Run the ring across two VMs and capture output demonstrating both hostnames.

15.2 Suggested report contents

Include your environment, source files, exact compilation/run commands, correctness checks, timing table, and interpretation. For a runtime plot, label the horizontal axis *Number of MPI processes* and the vertical axis *Execution time (seconds)*. For a speedup plot, label the vertical axis *Speedup* T_1/T_P . Separate native-Linux and VM results if you compare them.

15.3 Final self-check

1. Can one node run several MPI ranks?
2. Does a global C variable automatically become shared between ranks?
3. Does return from `MPI_Send` prove the receiver processed the data?
4. Why might send-first code work for a small message and hang later?
5. How do broadcast, scatter, gather, reduce, and all-reduce differ?
6. Which ranks must call a collective?
7. When can a nonblocking send buffer be modified again?
8. Why must timing comparisons keep the problem size fixed?

Answer guide. (1) Yes. (2) No. (3) No; an explicit protocol is needed to establish that. (4) Correctness may be depending on internal buffering. (5) Respectively: copy one input everywhere, distribute blocks, collect contributions, combine at a root, and combine everywhere. (6) All ranks in the specified communicator for these exercises. (7) After the send request has completed. (8) Otherwise the runs perform different amounts of work, so the strong-scaling speedup comparison would be misleading.